WEEK 12 · FINAL REVIEW & MOCK EXAM

Final Exam Preparation

45-Question Mock Exam · All 7 Domains

- Part 1: 7-Domain Master Summary (all key facts)
- Part 2: Exam Keyword Quick Reference
- Part 3: Mock Exam — 45 Questions (90 min simulation)
- Part 4: Answer Key with Full Explanations
- Part 5: Exam Day Cheat Sheet

45	90 min	~70%	USD 200
Questions	Duration	Pass Score	Cost

Table of Contents

PART 1 7-Domain Master Summary

Domain 1 — Platform Foundation (6%) 3

Domain 2 — Data Ingestion & Loading (21%) 4

Domain 3 — Transformation & Modeling (22%) 6

Domain 4 — Lakeflow Jobs (16%) 8

Domain 5 — CI/CD (10%) 9

Domain 6 — Troubleshooting & Optimization (10%) 10

Domain 7 — Governance & Security (15%) 11

PART 2 Exam Keyword Quick Reference

All Domains Keyword Map 13

PART 3 Mock Exam — 45 Questions

Questions 1–45 (90 min simulation) 15

PART 4 Answer Key & Explanations

Full explanations for all 45 questions 27

PART 5 Exam Day Cheat Sheet

One-page master reference 39

REVIEW

PART 1 — 7-Domain Master Summary

Essential facts for every domain

Domain 1 Databricks Intelligence Platform 6%

Topic	Key Facts
Lakehouse Architecture	Combines data lake (scale/cost) + warehouse (ACID/performance); Delta Lake is the foundation
Delta Lake	ACID transactions, time travel (VERSION AS OF / TIMESTAMP AS OF), schema enforcement, Z-ordering
Unity Catalog	3-level namespace: catalog.schema.table; single governance plane across all workspaces
Compute Types	All-purpose (interactive dev), Job cluster (ephemeral prod), SQL Warehouse (SQL tasks/BI), Serverless
Delta Time Travel	DESCRIBE HISTORY; SELECT … VERSION AS OF N; SELECT … TIMESTAMP AS OF "date"
VACUUM	Removes old files; default 7-day retention; cannot time-travel past vacuum point

Domain 2 Data Ingestion & Loading 21%

Method	Key Facts
COPY INTO	SQL batch load; idempotent (Delta log tracks files); COPY_OPTIONS("force"="true") re-loads; VALIDATE ALL dry-run
Auto Loader	cloudFiles streaming; cloudFiles.schemaLocation REQUIRED; _rescued_data for new columns; unique checkpointLocation per stream
Auto Loader modes	Directory listing (default, millions of files) vs File notification (cloudFiles.useNotifications=true, billions of files)
Schema evolution	rescue (default → _rescued_data) / merge (add to Delta) / fail (stop pipeline)
Triggers	trigger(availableNow=True) = process backlog then stop; trigger(once=True) deprecated
Lakeflow Connect	Standard connectors: Salesforce/Workday/ServiceNow (UI, no code); Managed: SAP/Oracle (partner, DLT)
JDBC	spark.read.format("jdbc"); parallel read needs partitionColumn+lowerBound+upperBound+numPartitions (all 4)
Semi-structured	raw:field (SQL, JSON string); col("a.b") (StructType); from_json() → StructType; explode() → rows

■ Domain 2 — Core Syntax

```
-- COPY INTO skeleton
COPY INTO target FROM "s3://bucket/path/" FILEFORMAT = PARQUET
FORMAT_OPTIONS ("mergeSchema"="true") COPY_OPTIONS ("mergeSchema"="true");

-- Auto Loader skeleton
spark.readStream.format("cloudFiles")
  .option("cloudFiles.format","json")
  .option("cloudFiles.schemaLocation","/chk/schema")
  .load("s3://bucket/path/")
  .writeStream.format("delta").option("checkpointLocation","/chk/stream")
  .trigger(availableNow=True).table("bronze.events")
```

Domain 3 Data Transformation & Modeling 22%

Topic	Key Facts
Null handling	dropna(subset,how,thresh); fillna(dict); coalesce(col1,col2,lit("default")); when/otherwise
Type casting	col("x").cast("double"); to_date(col,"yyyy-MM-dd") → NULL on bad input; to_timestamp()
Column ops	withColumn("existing") = overwrite; withColumnRenamed; drop(*list); filter/where
explode	explode() drops null/empty rows; explode_outer() keeps them; posexplode() adds index
MERGE INTO	WHEN MATCHED / NOT MATCHED / NOT MATCHED BY SOURCE (Databricks extension)
Join types	inner/left/right/full/left_semi(EXISTS)/left_anti(NOT EXISTS)/cross(cartesian)
Broadcast join	broadcast(df) hint; auto via autoBroadcastJoinThreshold=10MB; AQE auto-converts
Deduplication	dropDuplicates(keys) = arbitrary row; Window ROW_NUMBER() ORDER BY ts DESC = latest
Aggregations	approx_count_distinct(col,rsd) = fast HLL; countDistinct = exact/slow; summary() adds percentiles
union	unionByName(allowMissingColumns=True) = by name; union() = by POSITION (dangerous)
Gold objects	Delta Table=batch write; View=no storage; MV(DLT)=stored+auto-refresh; Streaming Table=incremental
DLT expectations	expect=warn/keep; expect_or_drop=drop row; expect_or_fail=stop pipeline
Spark tuning	shuffle.partitions=200 default; autoBroadcastJoinThreshold=10MB; AQE on by default
OPTIMIZE/ZORDER	OPTIMIZE=compact files; ZORDER BY(cols)=data co-location+skipping; max ~4 cols
Liquid Clustering	CLUSTER BY at creation; OPTIMIZE triggers incremental re-cluster; replaces ZORDER for new tables

Domain 4 Working with Lakeflow Jobs 16%

Topic	Key Facts
Task types	Notebook/SQL/Pipeline(DLT)/Python/dbt/Dashboard/Spark Submit/Run Job
SQL task	Requires SQL Warehouse — NOT a cluster
Cluster types	Job cluster=ephemeral+cheap(prod); All-purpose=persistent+expensive(dev)
depends_on	Defines DAG; default outcome=SUCCEEDED; outcome="FAILED"→error handler
run_if	ALL_SUCCESS(default)/AT_LEAST_ONE_SUCCESS/NONE_FAILED/ALL_DONE/AT_LEAST_ONE_FAILED/ALL_FAILED
Retry	max_retries; min_retry_interval_millis; retry_on_timeout; timeout_seconds
If/Else task	condition_task: op+left+right; branches via outcome="true"/"false"
For-Each task	inputs=JSON array; concurrency=N; {{input}}=current iteration value
Triggers	Scheduled(cron+timezone); File Arrival(path+debounce); Table Update(Delta log, ANY/ALL_UPDATED)
Task Values	dbutils.jobs.taskValues.set/get; {{tasks.X.values.key}} in conditions
Parameters	{{job.parameters.name}}; {{job.start_time.iso_date}}; dbutils.widgets.get()
Repair Run	Re-run only failed+skipped tasks; successful tasks keep results

Domain 5 Implementing CI/CD 10%

Topic	Key Facts
Git Folders	Integrated Git in workspace; clone/branch/commit/push/pull/PR; supported: GitHub/GitLab/ADO/Bitbucket
DABs	Declarative Automation Bundles = Databricks Asset Bundles; define resources as YAML code
databricks.yml	Root config: bundle name, workspace, variables, resources, targets
Variables	${var.name}; default at bundle level; override per target
targets	dev/staging/prod environments; each with workspace URL + variable overrides
mode: development	Auto-prefix resources [dev username]; PAUSE all job schedules
mode: production	Exact names; ACTIVE schedules; run_as service principal required
CLI commands	validate(syntax check); deploy(create/update); run(trigger); destroy(remove all)
deploy vs run	bundle deploy = create/update resources; bundle run = trigger job/pipeline
CI/CD order	validate → deploy → run → test
GitHub Actions	on: push/pull_request; secrets for DATABRICKS_HOST+TOKEN; env-specific deploy

Domain 6 Troubleshooting & Optimization 10%

Topic	Key Facts
Spark UI	Jobs/Stages/Tasks/SQL/Storage/Executors/Environment tabs
Skew signature	One task >> all others in Duration column; large median-to-max gap in summary stats
AQE	Auto-coalesces partitions, auto-broadcasts, handles skew — all ON by default
Skew fix	AQE first; then salting (random suffix to key; explode small table × N salts)
Spill (Memory)	Data size in memory before overflow; Spill (Disk) = bytes written to disk
OOM — Driver	collect()/toPandas() on large DF; fix: write() to storage instead
OOM — Executor	Task data > heap; fix: increase memory or increase shuffle.partitions
GC Time > 10%	Memory pressure warning → increase executor memory
Cluster PENDING	VM quota exceeded or spot instance unavailable
DESCRIBE DETAIL	numFiles, averageFileSize, clusteringColumns — diagnose small-file problem
Predictive Opt.	Auto-runs OPTIMIZE+VACUUM on UC managed tables; no manual schedule needed
repartition vs coalesce	repartition=full shuffle, balanced; coalesce=no shuffle, may create skew

Domain 7 Governance & Security 15%

Topic	Key Facts
UC hierarchy	metastore → catalog → schema → table/view/volume/function
3-privilege chain	USE CATALOG + USE SCHEMA + SELECT = minimum to read a table
Managed vs External	Managed DROP: deletes data+metadata; External DROP: metadata only (data preserved)
GRANT syntax	GRANT ON TO ``
REVOKE	Removes a granted privilege
DENY	Explicitly blocks access; overrides parent-level GRANTs
Column masking	CREATE FUNCTION → ALTER TABLE … ALTER COLUMN SET MASK fn; transparent to users
Row filter	CREATE FUNCTION (returns BOOLEAN) → ALTER TABLE … SET ROW FILTER fn ON (col)
is_account_group_member()	UC group check in mask/filter functions
current_user()	Returns querying user email in mask/filter functions
ABAC	Row filters + column masks using user attributes (group, session context)
Lineage	Auto-tracked by UC: table, column, notebook, job — no config needed
Audit logs	system.access.audit — all UC operations; query for SELECT/GRANT/REVOKE events
Volumes	Managed/External; READ VOLUME + WRITE VOLUME privileges; use in COPY INTO paths
Delta Sharing	CREATE SHARE + RECIPIENT + GRANT SELECT ON SHARE — share live Delta data externally

KEYWORDS

PART 2 — Exam Keyword Quick Reference

Scenario → Answer mapping

When you see this in a question…

If the question mentions…	The answer involves…
idempotent batch load from cloud storage	COPY INTO (Delta log tracks loaded files)
force re-ingest already-loaded files	COPY_OPTIONS ("force"="true")
dry-run validation before loading	COPY INTO … VALIDATE ALL
continuous streaming from cloud files	Auto Loader (cloudFiles format)
billions of files / low-latency events	Auto Loader file notification mode (useNotifications=true)
_rescued_data column	Auto Loader default schema rescue mode (new columns)
process backlog then stop (streaming)	trigger(availableNow=True)
Salesforce / Workday / ServiceNow, no code	Lakeflow Connect Standard Connector
SAP / Oracle / partner connector	Lakeflow Connect Managed Connector
JDBC parallel read 4 required options	partitionColumn + lowerBound + upperBound + numPartitions
JSON string column access in SQL	raw:field.subfield (colon operator)
flatten array to rows	explode() — drops null; explode_outer() — keeps null rows
Bronze → Silver cleaning	dropna/fillna/cast/withColumn/MERGE INTO pattern
cast() returns NULL silently	to_date() / to_timestamp() on unparseable input
keep latest duplicate per key	Window ROW_NUMBER() PARTITION BY key ORDER BY ts DESC, filter rn=1
arbitrary dedup, any row OK	dropDuplicates(["key_col"])
small table join optimization	broadcast() hint; auto-broadcast via autoBroadcastJoinThreshold
left side only, filter by match (EXISTS)	LEFT SEMI JOIN (how="left_semi")
filter by no match (NOT EXISTS)	LEFT ANTI JOIN (how="left_anti")
cartesian product	CROSS JOIN — dangerous on large tables
approximate distinct count	approx_count_distinct(col, rsd) — HyperLogLog
union by column name (safe)	unionByName() — union() matches by position (unsafe)
Gold auto-refresh aggregation	DLT Materialized View (@dlt.table)
continuous stream append to Delta	DLT Streaming Table
drop bad rows, continue pipeline	@dlt.expect_or_drop() / ON VIOLATION DROP ROW
stop pipeline on bad data	@dlt.expect_or_fail() / ON VIOLATION FAIL UPDATE
metric only, keep bad rows	@dlt.expect() — warn only
file compaction (small files)	OPTIMIZE
data skipping on filter column	ZORDER BY or Liquid Clustering CLUSTER BY
incremental auto-clustering (new tables)	Liquid Clustering — CLUSTER BY at creation
auto OPTIMIZE + VACUUM without manual jobs	Predictive Optimization (UC managed tables)

SQL task compute requirement	SQL Warehouse — NOT a cluster
cleanup task always runs	run_if="ALL_DONE"
error handler runs only on failure	depends_on outcome="FAILED"
loop over list of values in job	For-Each task with inputs array + concurrency
job runs when new file arrives	File Arrival trigger
job runs when Delta table changes	Table Update trigger (ANY_UPDATED)
define job/pipeline as YAML code	Databricks Asset Bundles (DABs) — databricks.yml
auto-pause schedules in dev env	mode: development in DABs target
deploy to specific environment	databricks bundle deploy --target
UC minimum privileges for table read	USE CATALOG + USE SCHEMA + SELECT (all three required)
DROP TABLE deletes data permanently	Managed table — both metadata and data files deleted
DROP TABLE keeps data files	External table — metadata only removed
hide SSN/email from non-admins	Column masking — ALTER TABLE … ALTER COLUMN SET MASK fn
users see only their region rows	Row Filter — ALTER TABLE … SET ROW FILTER fn ON (col)
block one table despite catalog grant	DENY SELECT ON TABLE to principal
access based on group membership + data attribute	ABAC pattern: row filter / column mask with is_account_group_member()
auto-tracked table dependencies	Unity Catalog data lineage (automatic, no config)
query all data access events	system.access.audit table
one task >> all others in duration	Data skew — AQE skew join or manual salting
Spill (Disk) in Stages tab	Disk spill — increase shuffle.partitions or executor memory
Driver OOM on collect()	Replace collect()/toPandas() with write() to storage
GC Time > 10% in Executors tab	Memory pressure — increase executor memory
cluster stuck in PENDING	VM quota exceeded or spot instance unavailable

MOCK EXAM

PART 3 — Mock Exam

45 Questions · 90 Minutes · All 7 Domains

■ **Exam Focus** Simulate exam conditions: set a 90-minute timer, work through all 45 questions without referring to notes, then check your answers in Part 4. Target score: 32/45 (≈71%) to pass.

Domain	Weight	# Questions in This Mock
1 — Platform Foundation	6%	3
2 — Data Ingestion & Loading	21%	9
3 — Transformation & Modeling	22%	10
4 — Lakeflow Jobs	16%	7
5 — CI/CD	10%	5
6 — Troubleshooting & Optimization	10%	5
7 — Governance & Security	15%	6
Total	100%	45

Instructions: Choose the single best answer for each question. 45 questions · 90 minutes · ~2 minutes per question. Flag uncertain questions and return to them.

Domain 1 — Platform

1. Which Delta Lake feature allows querying data as it existed at a specific point in the past?

A) Schema Evolution

B) Time Travel (VERSION AS OF / TIMESTAMP AS OF)

C) OPTIMIZE with ZORDER

D) Predictive Optimization

2. A data team runs VACUUM on a Delta table with default settings. After how many days of history will old data files be deleted?

A) 1 day

B) 7 days (168 hours — default retention)

C) 30 days

D) Files are never deleted by VACUUM

3. A SQL analytics workload runs in Databricks. Which compute type is most appropriate and cost-efficient?

A) All-purpose cluster — always available

B) Job cluster — ephemeral and cheaper

C) SQL Warehouse — optimized for SQL queries, auto-scales

D) Serverless notebook — no config needed

Domain 2 — Data Ingestion

4. A COPY INTO job is scheduled to run every hour on the same S3 path. A file loaded in the first run appears in the second run too. What happens?

A) The file is loaded again — COPY INTO is not idempotent

B) The file is skipped — COPY INTO tracks loaded files in the Delta transaction log

C) COPY INTO raises a DuplicateFileError

D) The file is loaded only if its content changed

5. An Auto Loader stream encounters a source JSON file with a new field "region" not in the current schema. Using default settings, where does "region" data go?

A) The stream fails with a schema mismatch error

B) The field is silently dropped

C) The field is added to the target table automatically

D) The field value is stored in the _rescued_data JSON column

6. Which cloudFiles option is REQUIRED when Auto Loader schema inference is enabled?

A) cloudFiles.inferColumnTypes

B) cloudFiles.schemaLocation

C) cloudFiles.schemaEvolutionMode

D) checkpointLocation

7. A team needs to ingest 2 trillion files from Azure ADLS Gen2 with sub-minute latency. Which Auto Loader mode should they use?

A) Directory listing mode (default)

B) File notification mode (cloudFiles.useNotifications=true)

C) COPY INTO with scheduled every-minute trigger

D) Lakeflow Connect Standard Connector

8. trigger(availableNow=True) is used in an Auto Loader writeStream. What happens after all existing files are processed?

A) The stream waits continuously for new files

B) The stream stops gracefully after the current backlog is processed

C) The stream fails with a TimeoutException

D) The stream switches to directory listing mode

9. A data engineer reads from a MySQL table with 200 million rows using JDBC. Only one Spark task is used. Which four options must ALL be set to enable parallel reads?

A) fetchSize, batchSize, parallelism, numWorkers

B) partitionColumn, lowerBound, upperBound, numPartitions

C) parallelReads=true, numPartitions, partitionKey, splitSize

D) partitionColumn and numPartitions are sufficient

10. A JSON payload is stored as a STRING column "raw". Which Databricks SQL syntax extracts the nested field user.address.city?

A) raw["user"]["address"]["city"]

B) raw:user.address.city

C) get_json_object(raw, "user.address.city")

D) json_extract(raw, "$.user.address.city")

11. Which Lakeflow Connect connector type handles Salesforce CRM data with NO custom code?

A) Managed Connector (partner-built DLT pipeline)

B) JDBC Connector with Salesforce driver

C) Standard Connector (UI-configured, Databricks-built)

D) REST API with Python requests

12. Two Auto Loader streams write to different Delta tables. An engineer uses the same checkpointLocation path for both. What is the result?

A) Both streams work — checkpoints are isolated by target table name

B) The second stream reuses the first stream's offset state — corrupt results

C) Databricks raises an error and prevents the duplicate checkpoint

D) The streams take turns — only one runs at a time

Domain 3 — Transformation

13. A Bronze table column "price" is stored as STRING. Which PySpark expression correctly casts it to DOUBLE?

A) df.withColumn("price", double(col("price")))

B) df.withColumn("price", col("price").cast("double"))

C) df.withColumn("price", to_double(col("price")))

D) df.cast(col("price"), DoubleType())

14. to_date(col("date_str"), "MM/dd/yyyy") is called on a row where date_str = "INVALID". What does the function return?

A) Throws a ParseException

B) Returns the string "INVALID"

C) Returns NULL silently

D) Returns the epoch date 1970-01-01

15. An engineer wants to keep only the most recently updated row per order_id. Which approach guarantees this?

A) df.dropDuplicates(["order_id"])

B) df.distinct()

C) Window ROW_NUMBER() PARTITION BY order_id ORDER BY updated_at DESC, filter rn=1

D) df.groupBy("order_id").agg(max("updated_at"))

16. Which join type returns only left-side rows that have NO matching row in the right table, with only left-side columns in the result?

A) LEFT OUTER JOIN where right columns are null

B) LEFT ANTI JOIN (how="left_anti")

C) LEFT SEMI JOIN (how="left_semi")

D) INNER JOIN with NOT IN filter

17. df_a.union(df_b) is called, but the two DataFrames have columns in a different order. What is the result?

A) Spark automatically matches columns by name

B) Columns are matched by POSITION — data ends up in wrong columns silently

C) Spark raises an AnalysisException about column order

D) The result DataFrame has duplicate columns

18. A dashboard requires pre-computed daily revenue aggregates that automatically refresh when upstream Silver data changes. Which Gold layer object is best?

A) Delta Table updated by a scheduled job

B) SQL View over Silver table

C) DLT Materialized View (@dlt.table)

D) Streaming Table with Auto Loader

19. @dlt.expect_or_drop("positive_amount", "amount > 0") is applied to a table. A row with amount = -10 arrives. What happens?

A) The pipeline fails and stops processing

B) The row is kept with amount set to NULL

C) The row is silently dropped; the violation count is recorded in metrics

D) The row is moved to a quarantine table

20. spark.sql.shuffle.partitions defaults to 200. A dataset is only 500MB. What is the best approach?

A) Manually set shuffle.partitions to 4 (500MB / 128MB target)

B) Leave at 200 — Adaptive Query Execution auto-coalesces small partitions

C) Set to 1000 for more parallelism

D) Disable AQE to prevent interference

21. OPTIMIZE ZORDER BY (customer_id, region) is run on a Delta table. A query filters WHERE customer_id = "C123" AND region = "US". What is the primary benefit?

A) Duplicate rows are removed

B) The query uses a B-tree index on customer_id

C) Related data is co-located in files, enabling Delta data skipping to skip most files

D) The table is repartitioned by customer_id

22. A Liquid Clustering table is created with CLUSTER BY (order_date). After thousands of inserts, clustering degrades. Which command re-clusters it?

A) OPTIMIZE orders ZORDER BY (order_date)

B) RECLUSTER orders ON (order_date)

C) OPTIMIZE orders — Liquid Clustering triggers automatically

D) DROP AND RECREATE the table with fresh clustering

Domain 4 — Lakeflow Jobs

23. A Lakeflow Job has tasks: Ingest → Transform → Publish. Transform fails. What happens to Publish?

A) Publish runs with Transform's partial output

B) Publish is skipped (not run) — default run_if="ALL_SUCCESS"

C) The entire job restarts from Ingest

D) Publish runs if Ingest succeeded, regardless of Transform

24. A task must run a SQL query against a Delta table. Which task type and compute are required?

A) Notebook task — any cluster type

B) SQL task — requires a SQL Warehouse

C) SQL task — runs on a job cluster

D) Python task — can run SQL via spark.sql()

25. A cleanup notebook should always run after the ETL task, whether ETL succeeded, failed, or timed out. Which run_if setting achieves this?

A) run_if="ALL_SUCCESS"

B) run_if="AT_LEAST_ONE_FAILED"

C) run_if="ALL_DONE"

D) run_if="NONE_FAILED"

26. A For-Each task has inputs=["US","EU","APAC"] and concurrency=2. How does execution proceed?

A) All 3 iterations run sequentially one by one

B) All 3 iterations run simultaneously

C) 2 iterations run in parallel; the 3rd waits until a slot is free

D) Each iteration runs on a separate cluster

27. A job should run whenever a new file appears in s3://bucket/incoming/orders/. Which trigger type should be configured?

A) Scheduled (cron) with 1-minute interval

B) File Arrival trigger monitoring the S3 path

C) Table Update trigger on the orders Delta table

D) Continuous trigger with processingTime="1 minute"

28. Task A stores a value: dbutils.jobs.taskValues.set("row_count", 500). Task B reads it. What dependency must exist?

A) No dependency — task values are globally accessible by all tasks

B) Task B must depend on Task A to guarantee the value is set before Task B reads it

C) A job parameter must be defined to forward the value

D) The value must be written to a Delta table to share between tasks

29. A failed job run has tasks: A (SUCCESS), B (FAILED), C (SKIPPED). The engineer uses Repair Run. Which tasks re-execute?

A) All tasks restart from A

B) Only B and C — A keeps its successful result

C) Only B — C is not re-run

D) Only C — the last task always reruns

Domain 5 — CI/CD

30. A bundle target is configured with mode: development. Which two behaviors are automatically applied?

A) Resources are deployed to production; schedules are active

B) Resources are prefixed with [dev username]; job schedules are paused

C) Resources are validated only — no actual deployment occurs

D) Cluster sizes are doubled for faster development

31. Which databricks CLI command validates the databricks.yml syntax without deploying anything?

A) databricks bundle check

B) databricks bundle validate

C) databricks bundle dry-run

D) databricks bundle lint

32. A bundle variable catalog_name has default "dev_catalog". The prod target sets catalog_name: "prod_catalog". What value does a notebook receive when deployed to prod?

A) "dev_catalog" — defaults always win

B) "prod_catalog" — target-level variables override bundle defaults

C) Both values — the notebook receives an array

D) An error — conflicting variable definitions are not allowed

33. What is the correct CI/CD operation order in a Databricks pipeline?

A) deploy → validate → run → test

B) validate → deploy → run → test

C) validate → test → deploy → run

D) run → validate → deploy → test

34. In databricks.yml, run_as under a prod target specifies what?

A) The user who created the bundle

B) The identity (service principal) under which production jobs execute

C) The cluster owner for job clusters

D) The Git user for commit attribution

Domain 6 — Troubleshooting

35. In the Spark UI Stages tab, one task takes 25 minutes while all others complete in under 30 seconds. What is the most likely cause and first fix?

A) shuffle.partitions is too low — increase to 2000

B) Data skew — one partition has far more data; AQE skew join handling or salting

C) The executor ran out of disk space — add storage

D) The task is waiting for an external API response

36. An executor's GC Time is 28% of total task time. What does this indicate and what should be done?

A) Normal — JVM GC always uses about 25-30%

B) Severe memory pressure — increase executor memory or reduce partition size

C) Disk I/O bottleneck — add faster storage

D) Network saturation — reduce shuffle data

37. A job fails with "java.lang.OutOfMemoryError" on the DRIVER node. What is the most common cause?

A) Executor memory is too small for the shuffle

B) df.collect() or df.toPandas() pulled a very large DataFrame to the driver

C) autoBroadcastJoinThreshold is set too high

D) shuffle.partitions is too low causing large executor tasks

38. A Delta table has 8,000 files with an average size of 2MB each. What is the problem and fix?

A) Too few files — run REPARTITION to create more

B) Small file problem — run OPTIMIZE to compact files to ~1GB target size

C) Files are too large — run SPLIT TABLE

D) Schema is corrupted — run REPAIR TABLE

39. What does Predictive Optimization automatically do for Unity Catalog managed tables?

A) Automatically runs ZORDER BY based on query patterns

B) Auto-runs OPTIMIZE (compaction) and VACUUM (cleanup) without manual scheduling

C) Automatically creates column statistics for the query optimizer

D) Predicts and pre-warms caches for upcoming queries

Domain 7 — Governance

40. A data analyst needs to read from prod_catalog.gold.summary. What is the minimum set of grants required?

A) SELECT on the table only

B) USE CATALOG on prod_catalog, USE SCHEMA on gold, SELECT on summary

C) USE CATALOG on prod_catalog and SELECT on summary

D) SELECT on the catalog cascades to all objects within it

41. DROP TABLE is executed on a MANAGED Delta table in Unity Catalog. What happens?

A) Only the metadata is deleted; data files remain in cloud storage

B) Both the metadata (UC) and the underlying data files are permanently deleted

C) The table is archived for 30 days before final deletion

D) The table is marked inactive; a PURGE command is needed to delete data

42. A column mask is applied to the "ssn" column. An analyst who is not in the "pii_admin" group queries SELECT * FROM customers. What do they see for the ssn column?

A) An access denied error

B) NULL for the ssn column

C) The masked value (e.g., ***-**-6789) as returned by the mask function

D) The ssn column is excluded from SELECT * results

43. A row filter function returns: is_account_group_member("finance") OR region = session_context("user_region"). A user in "finance" queries the table. How many rows do they see?

A) Only rows matching their user_region

B) All rows — is_account_group_member("finance") evaluates to TRUE for all rows

C) No rows — both conditions must be true

D) Only rows where region IS NOT NULL

44. An engineer runs: GRANT SELECT ON CATALOG prod_catalog TO `data_team`. Then: DENY SELECT ON TABLE prod_catalog.silver.pii TO `data_team`. What can data_team access?

A) Nothing — DENY on table blocks all catalog access

B) All tables in prod_catalog EXCEPT prod_catalog.silver.pii

C) Only prod_catalog.silver.pii — DENY inverts the grant

D) DENY is invalid if a GRANT exists at a higher level

45. Unity Catalog automatically captures which of the following without any configuration?

A) Query performance metrics and slow query warnings

B) Table and column lineage — which upstream tables/columns produced downstream objects

C) Data quality scores and anomaly alerts

D) Cost attribution by team or project

ANSWERS

PART 4 — Answer Key & Explanations

Full explanations for all 45 questions

■ **Exam Focus** Score yourself: 32+/45 = pass (~71%). For each wrong answer, note the domain and revisit the corresponding week's study guide.

Score Sheet

Q1: ___ (B)	Q2: ___ (B)	Q3: ___ (C)	Q4: ___ (B)	Q5: ___ (D)	Q6: ___ (B)	Q7: ___ (B)	Q8: ___ (B)	Q9: ___ (B)
Q10: ___ (B)	Q11: ___ (C)	Q12: ___ (B)	Q13: ___ (B)	Q14: ___ (C)	Q15: ___ (C)	Q16: ___ (B)	Q17: ___ (B)	Q18: ___ (C)
Q19: ___ (C)	Q20: ___ (B)	Q21: ___ (C)	Q22: ___ (C)	Q23: ___ (B)	Q24: ___ (B)	Q25: ___ (C)	Q26: ___ (C)	Q27: ___ (B)
Q28: ___ (B)	Q29: ___ (B)	Q30: ___ (B)	Q31: ___ (B)	Q32: ___ (B)	Q33: ___ (B)	Q34: ___ (B)	Q35: ___ (B)	Q36: ___ (B)
Q37: ___ (B)	Q38: ___ (B)	Q39: ___ (B)	Q40: ___ (B)	Q41: ___ (B)	Q42: ___ (C)	Q43: ___ (B)	Q44: ___ (B)	Q45: ___ (B)

Answers & Explanations

Q	Dom	Ans	Explanation
1	D1	(B)	Delta Lake time travel allows querying historical versions using VERSION AS OF or TIMESTAMP AS OF "". It works because Delta keeps transaction log entries for each version.
2	D1	(B)	VACUUM default retention is 7 days (168 hours). Files older than this threshold are permanently deleted. Time travel to removed versions becomes impossible after VACUUM.
3	D1	(C)	SQL Warehouses are optimized for SQL analytics workloads, support BI tools via JDBC/ODBC, and auto-scale. They are cost-efficient for SQL-heavy workloads and are required for SQL tasks in Lakeflow Jobs.
4	D2	(B)	COPY INTO is idempotent by default. It records loaded files in the Delta transaction log. On subsequent runs, already-loaded files are skipped unless COPY_OPTIONS("force"="true") is specified.
5	D2	(D)	Auto Loader default schema evolution mode is "rescue" — new fields go into _rescued_data as JSON. The stream continues without failure. Use mergeSchema=true on writeStream to add new fields to the target table.
6	D2	(B)	cloudFiles.schemaLocation is required when schema inference is active. It persists the inferred schema to a storage path so it survives stream restarts. Without it, schema is re-inferred from scratch on each restart.
7	D2	(B)	File notification mode uses Azure Event Grid subscriptions to detect new files instantly without scanning the directory. It scales to billions/trillions of files — directory listing would be too slow at that scale.
8	D2	(B)	trigger(availableNow=True) processes all data available at stream startup, then stops cleanly. It replaces the deprecated trigger(once=True) and enables batch-style streaming for scheduled pipelines.
9	D2	(B)	Parallel JDBC reads require ALL FOUR: partitionColumn (numeric/date column), lowerBound (min value), upperBound (max value), numPartitions (number of parallel tasks). Missing any one falls back to single-partition read.
10	D2	(B)	The colon (:) operator is Databricks SQL native syntax for JSON string extraction with dot notation for nesting. get_json_object() also works but requires the $ prefix JSONPath syntax.

11	D2	(C)	Salesforce is supported by Lakeflow Connect Standard Connectors — configured entirely in the Databricks UI with no code. Standard connectors create DLT pipelines internally and handle auth, incremental sync, and schema mapping.
12	D2	(B)	Each stream MUST have a unique checkpointLocation. Sharing checkpoints between streams corrupts offset tracking and file state, leading to incorrect data in both targets. This is a critical best practice.
13	D3	(B)	cast() is a method called on a Column object: col("price").cast("double"). double() and to_double() are not valid PySpark functions. The DoubleType() form also works: col("price").cast(DoubleType()).
14	D3	(C)	to_date() returns NULL when it cannot parse the input — it never throws an exception by default. Always validate after parsing: df.filter(col("date_col").isNull()).count() to find parsing failures.
15	D3	(C)	dropDuplicates() keeps an arbitrary row — no ordering guarantee. Window ROW_NUMBER() with ORDER BY updated_at DESC assigns rank 1 to the most recent row per key, then filtering rn=1 keeps exactly that row.
16	D3	(B)	LEFT ANTI JOIN returns left-side rows with NO match in the right — equivalent to WHERE NOT EXISTS. LEFT SEMI returns rows WITH a match (opposite). Both return only left-side columns.
17	D3	(B)	union() matches columns by POSITION, not name. If column order differs, data silently lands in wrong columns. Always use unionByName() to match by name safely.
18	D3	(C)	DLT Materialized Views are stored (physical Delta files) and auto-refreshed by the DLT pipeline when upstream source data changes. SQL Views have no storage and re-compute on every query — too slow for dashboards.
19	D3	(C)	expect_or_drop removes violating rows from the output table and records the violation count in DLT metrics. The pipeline continues normally. Use expect_or_fail to stop the pipeline on violations.
20	D3	(B)	With AQE enabled (Databricks default), spark.sql.adaptive.coalescePartitions automatically merges small shuffle partitions at runtime. For 500MB, AQE coalesces 200 partitions down to ~4. No manual change needed.
21	D3	(C)	ZORDER co-locates rows with the same customer_id and region in the same files. Delta's min/max statistics allow Spark to skip entire files that don't contain the queried values — dramatically reducing I/O.
22	D3	(C)	For Liquid Clustering tables, OPTIMIZE (without ZORDER BY) triggers incremental re-clustering using the CLUSTER BY columns defined at table creation. No column specification is needed.
23	D4	(B)	Default run_if="ALL_SUCCESS" means a task runs only if ALL upstream dependencies succeeded. If Transform fails, Publish is skipped and the job run is marked FAILED.
24	D4	(B)	The SQL task type specifically requires a SQL Warehouse — it cannot run on a cluster. Notebook tasks can run SQL code on a cluster, but the SQL task type itself needs a warehouse.
25	D4	(C)	run_if="ALL_DONE" runs the task after all upstream tasks have finished, regardless of their outcome (success, failure, timeout, or skipped). Perfect for cleanup or notification tasks.
26	D4	(C)	concurrency=2 means up to 2 child task iterations run simultaneously. With 3 inputs, US and EU start in parallel; APAC waits until either US or EU completes and a slot becomes available.
27	D4	(B)	File Arrival trigger monitors a cloud storage path and fires the job when new files appear. It includes built-in deduplication so the same file won't trigger twice, and supports min_time_between_triggers_seconds debouncing.

28	D4	(B)	Task B must have depends_on Task A. Without this dependency, the tasks might run in parallel, and Task B could attempt to read the value before Task A has set it.
29	D4	(B)	Repair Run re-runs only failed and skipped tasks. Task A already succeeded, so it keeps its result. Tasks B (failed) and C (skipped because B failed) are re-run from their starting point.
30	D5	(B)	mode: development auto-prefixes all resource names with [dev] to prevent conflicts with production resources, and automatically PAUSES all job schedules to prevent accidental runs.
31	D5	(B)	databricks bundle validate checks databricks.yml for syntax errors and schema violations without creating or modifying any workspace resources. Always run this in CI before deploying.
32	D5	(B)	Target-level variable values override bundle-level defaults. When deploying to the prod target, catalog_name resolves to "prod_catalog". This is the key mechanism for environment differentiation in DABs.
33	D5	(B)	Standard CI/CD order: validate (syntax check, no changes) → deploy (create/update resources) → run (trigger job/pipeline) → test (verify results). Always validate before deploying to catch config errors early.
34	D5	(B)	run_as under a target specifies the execution identity for all jobs in that environment. For production, a service principal (not a human user) should be specified to ensure jobs run reliably without dependency on individual user accounts.
35	D6	(B)	A single outlier task is the classic data skew symptom. One key value (e.g., NULL or a dominant customer_id) has disproportionately more data in its partition. First check: AQE skew join (enabled by default). If insufficient: manual salting.
36	D6	(B)	GC Time > 10% signals memory pressure. At 28%, the JVM is spending excessive time garbage-collecting, indicating executor heap is nearly full. Fix: increase executor memory, raise shuffle.partitions to reduce per-task data size, or unpersist unused cached DataFrames.
37	D6	(B)	Driver OOM occurs when data is collected to the driver node. collect() and toPandas() bring the entire DataFrame into driver memory. Fix: write the DataFrame to storage with df.write.format("delta").save() instead of collecting it.
38	D6	(B)	8,000 × 2MB = 16GB in tiny files. The small-file problem causes excessive metadata overhead and slow reads. OPTIMIZE compacts small files into ~1GB target files, dramatically reducing file count and improving query performance.
39	D6	(B)	Predictive Optimization automatically runs OPTIMIZE (compact small files) and VACUUM (remove old files) on Unity Catalog managed tables based on usage patterns and statistics — eliminating the need for manual maintenance jobs.
40	D7	(B)	Unity Catalog requires the full privilege chain: USE CATALOG on the catalog + USE SCHEMA on the schema + SELECT on the specific table. Missing any one of the three grants will block access.
41	D7	(B)	DROP TABLE on a managed table permanently deletes BOTH the Unity Catalog metadata AND the underlying data files in managed storage. This is irreversible (unless within the Delta time travel retention window).
42	D7	(C)	Column masks are transparent — SELECT * works normally and the ssn column appears in results, but its value passes through the mask function. Non-admins see the masked representation; the query does not fail.
43	D7	(B)	The filter uses OR — if is_account_group_member("finance") is TRUE, the entire condition is TRUE for every row (short-circuit). Finance group members see ALL rows. Other users see only rows where region matches their session context.

44	D7	(B)	DENY explicitly blocks access to a specific object and overrides the parent-level GRANT. data_team has SELECT on all tables in prod_catalog (from the catalog GRANT), but the DENY on pii blocks that specific table only.
45	D7	(B)	Unity Catalog automatically captures table and column lineage — tracking which upstream tables, columns, notebooks, and jobs contributed to downstream objects. This requires no configuration and is visible in Catalog Explorer.

Domain Performance Analysis

After scoring, count wrong answers per domain to identify weak areas:

Domain	# Questions	Your Score	Status	If Below 60% — Review
1 — Platform	3	/ 3		Weeks 1–2: Delta Lake, UC basics, compute types
2 — Ingestion	9	/ 9		Weeks 3–5: COPY INTO, Auto Loader, cloudFiles options
3 — Transformation	10	/ 10		Weeks 6–8: ETL, joins, dedup, Gold objects, DLT
4 — Lakeflow Jobs	7	/ 7		Week 9: task types, run_if, triggers, For-Each
5 — CI/CD	5	/ 5		Week 10: DABs, databricks.yml, targets, CLI
6 — Troubleshooting	5	/ 5		Week 11a: Spark UI, skew, OOM, OPTIMIZE
7 — Governance	6	/ 6		Week 11b: GRANT/REVOKE, masking, RLS, ABAC
TOTAL	45	/ 45		Pass: 32+ (≈71%)

CHEAT SHEET

PART 5 — Exam Day Cheat Sheet

Final reference before entering the exam

Domain 1 — Platform (6%)

Key Fact	Remember
Delta time travel	VERSION AS OF N / TIMESTAMP AS OF "date"
VACUUM default	7 days (168h); cannot time-travel past vacuum
Compute for SQL tasks	SQL Warehouse — NOT a cluster
Job cluster vs all-purpose	Job=ephemeral+cheap(prod); All-purpose=persistent+expensive(dev)

Domain 2 — Data Ingestion (21%)

Key Fact	Remember
COPY INTO idempotency	Delta log tracks files; skip already-loaded; force=true to override
Auto Loader required option	cloudFiles.schemaLocation REQUIRED with inference
Auto Loader _rescued_data	Default rescue mode — new columns go here; stream continues
File notification trigger	cloudFiles.useNotifications=true; scales to billions of files
trigger(availableNow=True)	Process backlog then stop (replaces deprecated once=True)
JDBC parallel 4 options	partitionColumn + lowerBound + upperBound + numPartitions (ALL required)
JSON string operator	raw:field.sub (colon = SQL; col("a.b") = StructType)
Lakeflow Connect	Standard=Salesforce/SaaS no-code; Managed=SAP/Oracle partner

Domain 3 — Transformation (22%)

Key Fact	Remember
cast() syntax	col("x").cast("double") — NOT cast(col("x"),"double")
to_date() on bad input	Returns NULL silently — always validate after
dropDuplicates vs ROW_NUMBER	dropDuplicates=arbitrary; ROW_NUMBER=deterministic (keep latest)
LEFT SEMI / LEFT ANTI	SEMI=EXISTS filter(left cols only); ANTI=NOT EXISTS filter
union vs unionByName	union=by position (dangerous); unionByName=by name (safe)
DLT expectations	expect=warn/keep; expect_or_drop=drop; expect_or_fail=STOP
Gold: MV vs View	MV=stored+auto-refresh; View=no storage, re-computes every query
ZORDER vs Liquid	ZORDER=explicit+full rewrite; Liquid=CLUSTER BY+incremental
Predictive Opt.	Auto OPTIMIZE+VACUUM on UC managed tables; no manual job needed

Domain 4 — Lakeflow Jobs (16%)

Key Fact	Remember
SQL task compute	SQL task = SQL Warehouse ONLY
run_if ALL_DONE	Always runs — success/fail/timeout/skip upstream
outcome FAILED	depends_on outcome="FAILED" = error handler pattern
For-Each concurrency	concurrency=N; {{input}}=current value; default=1 (sequential)
Triggers	Scheduled(cron); File Arrival(path+debounce); Table Update(Delta log)
Task Values	dbutils.jobs.taskValues.set/get; Task B must depend on Task A

Repair Run	Re-runs only FAILED+SKIPPED; SUCCEEDED tasks keep results

Domain 5 — CI/CD (10%)

Key Fact	Remember
databricks.yml	Required root file; bundle/variables/resources/targets
mode: development	Prefix [dev user] + PAUSE schedules
mode: production	Exact names + ACTIVE schedules + run_as SP required
CLI order	validate → deploy → run → test
deploy vs run	deploy=create/update resources; run=trigger job/pipeline
${var.name}	Bundle variable; target-level overrides bundle default

Domain 6 — Troubleshooting (10%)

Key Fact	Remember
Skew signature	One task >> all others in duration; AQE or salting to fix
Driver OOM	collect()/toPandas() on large DF → write() instead
Executor OOM	Task > heap; increase memory or raise shuffle.partitions
GC Time > 10%	Memory pressure warning → more executor memory
Spill (Disk)	Overflow to disk; raise shuffle.partitions or executor memory
Cluster PENDING	VM quota exceeded or spot unavailable; switch to on-demand
Small-file fix	OPTIMIZE ; DESCRIBE DETAIL to check numFiles

Domain 7 — Governance (15%)

Key Fact	Remember
3-privilege chain	USE CATALOG + USE SCHEMA + SELECT = minimum for table read
Managed DROP	Deletes metadata + data files (PERMANENT)
External DROP	Deletes metadata ONLY; data files survive
DENY precedence	Overrides parent GRANT for specific principal on specific object
Column mask	ALTER COLUMN SET MASK fn; transparent, returns masked value
Row filter	SET ROW FILTER fn ON (col); fn returns BOOLEAN; true=visible
is_account_group_member()	UC group check in mask/filter; use over is_member()
ABAC in Databricks	Row filters + column masks with user attribute checks
Audit log	system.access.audit — all UC operations; no extra config needed

TIPS

Exam Day Tips

Final advice before you take the exam

✦ **Time management:** 45 questions in 90 minutes = 2 minutes/question. Flag unsure questions and return. Never leave blanks — guess if needed (no penalty).

✦ **Domain weights:** Focus time on Domain 3 (22%) + Domain 2 (21%) + Domain 7 (15%) = 58% of exam. Nail these three.

✦ **High-frequency topics:** COPY INTO idempotency, Auto Loader cloudFiles options, Window ROW_NUMBER dedup, GRANT privilege chain, DLT expectations — appear most often.

✦ **Elimination strategy:** Eliminate obviously wrong options first. Two answers often differ by one word ("cluster" vs "warehouse", "managed" vs "external") — focus there.

✦ **Code questions:** For syntax questions, mentally trace through the code. Watch for: cast() method order, union vs unionByName, depends_on outcome values.

✦ **AQE is default ON:** Many optimization questions have "AQE handles this automatically" as the right answer. Databricks enables AQE by default.

✦ **SQL Warehouse:** SQL task ≠ cluster. Every question about running SQL queries in a job → SQL Warehouse.

✦ **DROP TABLE behavior:** Managed = data gone. External = data stays. This appears repeatedly in governance questions.

Good luck on your Databricks Certified Data Engineer Associate exam! You have covered all 7 domains across 12 weeks of structured study. Trust your preparation.

PATTERNS **PART 6 — Essential SQL & PySpark Patterns** Memorize these before exam day

A. Data Ingestion Patterns

■ Ingestion SQL/PySpark patterns

```
-- COPY INTO: batch idempotent load
COPY INTO catalog.bronze.orders
FROM "s3://bucket/raw/orders/"
FILEFORMAT = PARQUET
FORMAT_OPTIONS ("mergeSchema"="true")
COPY_OPTIONS ("mergeSchema"="true");

-- COPY INTO: force re-load
COPY INTO catalog.bronze.orders FROM "s3://bucket/raw/"
FILEFORMAT = JSON COPY_OPTIONS ("force"="true");

-- Auto Loader: minimal pattern
(spark.readStream.format("cloudFiles")
 .option("cloudFiles.format","json")
 .option("cloudFiles.schemaLocation","/chk/schema")
 .load("s3://bucket/raw/")
 .writeStream.format("delta")
 .option("checkpointLocation","/chk/events")
 .trigger(availableNow=True)
 .table("catalog.bronze.events"))
```

B. Bronze → Silver ETL Patterns

■ ETL transformation pattern

```
from pyspark.sql.functions import col, to_date, upper, trim, coalesce, lit, current_timestamp

df = (spark.table("catalog.bronze.orders")
  .dropna(subset=["order_id","customer_id"])
  .withColumn("amount",     col("amount_str").cast("double"))
  .withColumn("order_date", to_date(col("date_str"), "yyyy-MM-dd"))
  .withColumn("status",     upper(trim(col("status"))))
  .fillna({"region":"UNKNOWN"})
  .withColumn("_silver_ts", current_timestamp())
  .drop("amount_str","date_str")
)

# MERGE INTO: upsert to Silver
from delta.tables import DeltaTable
DeltaTable.forName(spark,"catalog.silver.orders").alias("t")
  .merge(df.alias("s"), "t.order_id = s.order_id")
  .whenMatchedUpdateAll().whenNotMatchedInsertAll().execute()
```

C. Window & Deduplication Patterns

■ Window function patterns

```
from pyspark.sql.functions import row_number, rank, sum as spark_sum, lag
from pyspark.sql.window import Window

# Keep latest per key
w = Window.partitionBy("order_id").orderBy(col("updated_at").desc())
df.withColumn("rn", row_number().over(w)).filter(col("rn")==1).drop("rn")

# Running total
```

```
w_cum = Window.partitionBy("customer_id").orderBy("order_date")
              .rowsBetween(Window.unboundedPreceding, Window.currentRow)
df.withColumn("running_total", spark_sum("amount").over(w_cum))

# Lag (previous row value)
w_lag = Window.partitionBy("customer_id").orderBy("order_date")
df.withColumn("prev_amount", lag("amount", 1).over(w_lag))
```

D. Join Patterns

■ **Join patterns**

```
from pyspark.sql.functions import broadcast

# Standard joins
df_orders.join(df_customers, on="customer_id", how="inner")
df_orders.join(df_customers, on="customer_id", how="left")
df_orders.join(df_customers, on="customer_id", how="left_semi")
df_orders.join(df_customers, on="customer_id", how="left_anti")

# Broadcast join for small table
df_orders.join(broadcast(df_country), on="country_code", how="left")

-- SQL broadcast hint
SELECT /*+ BROADCAST(c) */ o.*, c.name
FROM silver.orders o JOIN silver.country c ON o.country_code = c.code;

# Multi-key join
df_a.join(df_b, on=["country_code","product_id"], how="inner")
```

E. Unity Catalog Privilege Patterns

■ **Unity Catalog SQL patterns**

```
-- Minimum to read a table
GRANT USE CATALOG ON CATALOG prod    TO `analysts`;
GRANT USE SCHEMA  ON SCHEMA prod.silver TO `analysts`;
GRANT SELECT      ON TABLE prod.silver.orders TO `analysts`;

-- Block one table despite broader grant
GRANT SELECT ON CATALOG prod TO `analysts`;   -- broad grant
DENY  SELECT ON TABLE prod.silver.pii TO `analysts`;  -- specific block

-- Column mask setup
CREATE FUNCTION mask_ssn(ssn STRING)
RETURN CASE WHEN is_account_group_member("pii_admin") THEN ssn
  ELSE CONCAT("***-**-", RIGHT(ssn,4)) END;
ALTER TABLE prod.silver.customers ALTER COLUMN ssn SET MASK mask_ssn;

-- Row filter setup
CREATE FUNCTION region_filter(region STRING)
RETURN is_account_group_member("global_team") OR region = session_context("user_region");
ALTER TABLE prod.silver.orders SET ROW FILTER region_filter ON (region);
```

TRAPS

PART 7 — Common Exam Traps & Confusables

Distinctions that trip up candidates

Trap	Wrong Answer	Correct Answer
COPY INTO re-running same files	Loads files again (not idempotent)	Skips already-loaded files (idempotent); force=true to override
Auto Loader on bad schema row	Stream fails	Row goes to _rescued_data; stream continues (rescue mode default)
cloudFiles option prefix	cloudFiles.header = "true"	header = "true" — format options have NO cloudFiles. prefix
Two streams, same checkpoint	Works fine	Corrupts both streams; each stream needs UNIQUE checkpointLocation
trigger(once=True) current status	Still valid	DEPRECATED — use trigger(availableNow=True) instead
JDBC parallel read: 2 options	partitionColumn + numPartitions sufficient	All FOUR required: partitionColumn+lowerBound+upperBound+numPartitions
union() column matching	Matches by column name	Matches by POSITION — use unionByName() for name-based matching
dropDuplicates keeps which row	Most recent row (latest timestamp)	Arbitrary row — use Window ROW_NUMBER() for deterministic dedup
to_date() on bad format	Throws ParseException	Returns NULL silently — always validate after casting
withColumn("existing",expr)	Creates a new duplicate column	Overwrites (replaces) the existing column
explode() on null array	Row kept with null item	Row DROPPED — use explode_outer() to keep null/empty array rows
LEFT SEMI columns in result	Both left and right columns	Left columns ONLY — that is the key difference from INNER JOIN
DLT expect vs expect_or_drop	expect drops bad rows	expect=warn only (keeps rows); expect_or_drop=drops rows
Managed table DROP	Only metadata deleted	Both metadata AND data files permanently deleted
External table DROP	Both deleted	Only metadata deleted; data files REMAIN at LOCATION path
USE CATALOG enough to read table	Yes — cascades to tables	NO — also need USE SCHEMA + SELECT on table (3-privilege chain)
DENY removable with REVOKE	REVOKE removes all	REVOKE DENY is needed to remove a DENY (not just REVOKE)
mode:development schedules	Schedules stay active	Schedules are PAUSED automatically
databricks bundle run action	Deploys resources	TRIGGERS the job/pipeline; deploy creates/updates resources
SQL task cluster type	Runs on job cluster	SQL task requires SQL WAREHOUSE — not a cluster

run_if ALL_DONE vs NONE_FAILED	Same thing	ALL_DONE=runs regardless; NONE_FAILED=skips if any upstream failed
GC Time > 10% cause	Disk bottleneck	Memory pressure — executor heap nearly full; OOM risk
Cluster PENDING = startup issue	Init script error (would show in logs)	VM quota or spot instance unavailable is most common cause
AQE status in Databricks	Manual opt-in required	ON BY DEFAULT — handles skew, coalescing, broadcast automatically

FLASH CARDS PART 8 — Last-Minute Flash Cards

Read through on exam morning

COPY INTO tracks files via	Delta transaction log	**Auto Loader tracks files via**	Checkpoint directory (RocksDB)
cloudFiles.schemaLocation	Required — persists inferred schema	**_rescued_data column**	New columns not in schema (rescue mode)
trigger(availableNow=True)	Process backlog then STOP	**File notification requires**	cloudFiles.useNotifications = true
JDBC needs 4 options for parallel	partitionColumn+lowerBound+upperBound+numPartitions	**raw:field in Databricks SQL**	Extract from JSON STRING column
from_json() returns	StructType (typed struct)	**get_json_object() returns**	STRING (single field)
explode() on null array	Row DROPPED	**explode_outer() on null array**	Row KEPT (null in output)
cast("double") on bad string	Returns NULL silently	**dropDuplicates() guarantee**	Arbitrary row — no ordering
Window ROW_NUMBER guarantee	Deterministic by ORDER BY	**LEFT SEMI columns**	Left side ONLY
LEFT ANTI returns	Rows with NO match	**CROSS JOIN result size**	Left rows × Right rows
union() matching strategy	By POSITION (dangerous)	**unionByName() matching strategy**	By COLUMN NAME (safe)
DLT expect behavior	Warn + keep rows	**DLT expect_or_drop behavior**	Drop rows, continue
DLT expect_or_fail behavior	STOP pipeline	**Gold MV vs SQL View**	MV=stored; View=no storage, recomputes
Spark shuffle.partitions default	200 (AQE auto-coalesces)	**autoBroadcastJoinThreshold default**	10MB
AQE status in Databricks	ON by default	**ZORDER max practical columns**	~4
Liquid Clustering vs ZORDER	Liquid=incremental; ZORDER=full rewrite	**OPTIMIZE on Liquid table**	Triggers incremental re-cluster
Predictive Optimization does	Auto OPTIMIZE + VACUUM	**Cluster PENDING cause**	VM quota or spot unavailable
GC Time > 10% means	Memory pressure warning	**Driver OOM cause**	collect()/toPandas() on large DF
SQL task compute requirement	SQL WAREHOUSE (not cluster)	**Job cluster vs all-purpose**	Job=ephemeral+cheap; All-purpose=persistent+expensive
run_if ALL_DONE	Always runs regardless of outcome	**outcome="FAILED" on depends_on**	Error handler pattern

For-Each {{input}}	Current iteration value	**File Arrival trigger debounce**	min_time_between_triggers_seconds
mode: development does	Prefix resources + PAUSE schedules	**mode: production requires**	run_as service principal
databricks bundle validate	Syntax check — NO deployment	**databricks bundle deploy**	Create/update resources
databricks bundle run	TRIGGER job/pipeline	**CI/CD order**	validate → deploy → run → test
UC 3-privilege chain	USE CATALOG + USE SCHEMA + SELECT	**Managed table DROP**	Deletes metadata + DATA (permanent)
External table DROP	Deletes metadata ONLY (data stays)	**DENY vs REVOKE**	DENY blocks; REVOKE removes grant
is_account_group_member()	UC group check in mask/filter	**Column mask applied with**	ALTER TABLE … ALTER COLUMN SET MASK
Row filter applied with	ALTER TABLE … SET ROW FILTER fn ON (col)	**Row filter function returns**	BOOLEAN — true = row visible
ABAC in Databricks	Row filters + masks with user attributes	**UC lineage tracking**	Automatic — no config needed
UC audit log table	system.access.audit		

BONUS QS

PART 9 — Additional Practice Questions

15 bonus questions across all domains

B1. Which Auto Loader trigger mode processes all backlog files then stops, suitable for scheduled pipelines?

A) trigger(processingTime="0 seconds")

B) trigger(once=True) (deprecated)

C) trigger(availableNow=True)

D) trigger(continuous=True)

B2. A DLT pipeline uses APPLY CHANGES INTO. What is the role of SEQUENCE BY?

A) Defines the output partition order

B) Specifies the column that determines which change event is most recent for a given key

C) Sets the watermark for late data handling

D) Controls the order rows are inserted into the streaming table

B3. coalesce(col("email_primary"), col("email_secondary"), lit("unknown")) returns what when email_primary IS NULL and email_secondary = "b@x.com"?

A) NULL

B) "unknown"

C) "b@x.com"

D) An error — coalesce requires matching types

B4. An engineer accidentally ran DROP TABLE on a managed Delta table. The table had 5 days of Delta history. Is recovery possible?

A) No — managed table DROP permanently deletes data files

B) Yes — use RESTORE TABLE to roll back to a previous version

C) Yes — use UNDROP TABLE to restore within 30 days

D) Yes — data is in the recycle bin for 7 days

B5. Which function in PySpark adds a column containing the file path that each row was read from?

A) current_file()

B) input_file_name()

C) source_path()

D) file_location()

B6. A bundle is deployed with: databricks bundle deploy --target staging. Then databricks bundle run my_job --target staging. What does the run command do?

A) Re-deploys the bundle to staging

B) Triggers the job defined as my_job in the bundle on the staging workspace

C) Validates the bundle configuration

D) Removes the deployed resources from staging

B7. posexplode(col("items")) differs from explode(col("items")) in which way?

A) posexplode keeps null/empty array rows; explode drops them

B) posexplode returns (index, element) pairs with the array position; explode returns element only

C) posexplode is SQL-only; explode is PySpark-only

D) posexplode creates a struct; explode returns a scalar

B8. A table has CLUSTER BY (customer_id) set at creation. An engineer runs: OPTIMIZE my_table ZORDER BY (customer_id). What is the result?

A) Both Liquid Clustering and ZORDER are applied — doubled performance

B) ZORDER overrides the CLUSTER BY definition going forward

C) This is incorrect — Liquid Clustering and ZORDER should not be combined; OPTIMIZE without ZORDER is correct

D) The table is re-partitioned by customer_id

B9. Which privilege is required to create a new Delta table inside an existing schema in Unity Catalog?

A) CREATE TABLE on the schema

B) MODIFY on the schema

C) SELECT on the schema

D) CREATE TABLE on the catalog

B10. An If/Else task uses op="LESS_THAN", left="{{tasks.count_task.values.rows}}", right="1000". Task A depends on outcome="true". Task B depends on outcome="false". When rows=500, which runs?

A) Task A (true branch) — 500 < 1000 is TRUE

B) Task B (false branch) — 500 < 1000 is FALSE

C) Both Task A and Task B run in parallel

D) Neither — LESS_THAN requires numeric type

B11. Which command removes ALL deployed resources from a target workspace for a DAB?

A) databricks bundle delete --target prod

B) databricks bundle destroy --target prod

C) databricks bundle remove --target prod

D) databricks bundle undeploy --target prod

B12. A DLT streaming table uses Auto Loader as the source. After a pipeline restart, which files are reprocessed?

A) All files in the source path are reprocessed from the beginning

B) Only files arrived since the last pipeline run

C) Only files that previously failed to process

D) DLT manages checkpoints — no files are reprocessed; processing resumes from where it stopped

B13. summary() is called on a DataFrame. Which statistics does it return that describe() does NOT?

A) mean and count

B) min and max

C) Percentiles (25%, 50%, 75%)

D) stddev

B14. A row filter function is defined and applied to a table. A user runs EXPLAIN on a query against that table. What happens to the filter?

A) The filter is visible in the query plan as a separate FilterExec node

B) The filter is applied transparently — users cannot see or bypass it

C) EXPLAIN skips row filters to show the raw plan

D) The user receives an error if they do not have is_account_group_member permission

B15. A streaming job using Auto Loader fails midway through processing a batch. The same stream is restarted with the identical checkpointLocation. What happens?

A) Processing starts from the beginning of the source path

B) Processing resumes from the last committed offset — no data is lost or duplicated

C) The checkpoint is invalidated and must be deleted before restarting

D) Processing skips to the most recently arrived file

Bonus Questions — Answer Key

#	Ans	Explanation
B1.	(C)	trigger(availableNow=True) processes all available data at stream startup then stops cleanly. trigger(once=True) had the same behavior but is deprecated. Use availableNow=True for new code.
B2.	(B)	SEQUENCE BY specifies the column (typically a timestamp or version) used to order CDC events for the same key. The event with the highest SEQUENCE BY value is treated as the latest state for that key.
B3.	(C)	coalesce() returns the first non-null value from left to right. email_primary is null, so it moves to email_secondary = "b@x.com" and returns that. "unknown" would only be returned if both are null.
B4.	(A)	DROP TABLE on a managed table permanently deletes the data files. There is no UNDROP or recycle bin. RESTORE TABLE only works on existing tables — you cannot restore a dropped managed table.
B5.	(B)	input_file_name() is a PySpark function that returns the full path of the file from which each row was read. Commonly used in Auto Loader to add _source_file metadata to bronze tables.
B6.	(B)	bundle run triggers (executes) the job or pipeline identified by the resource key in the bundle on the specified target workspace. It does NOT deploy — deploy is a separate step.
B7.	(B)	posexplode() returns two columns: the integer position (0-based index) and the element value. explode() returns only the element. Use posexplode when you need to know where each element was in the original array.
B8.	(C)	For Liquid Clustering tables, run OPTIMIZE without ZORDER BY — that triggers incremental clustering. Running OPTIMIZE ZORDER BY on a Liquid Clustering table mixes two approaches and is not recommended.
B9.	(A)	CREATE TABLE privilege must be granted on the SCHEMA (not the catalog) to allow creating new tables within that schema. MODIFY is for writing data to existing tables, not creating new ones.
B10.	(A)	LESS_THAN: 500 < 1000 is TRUE. Task A (outcome="true") runs. Task B (outcome="false") is skipped. The If/Else task evaluates the condition and routes to the appropriate branch.
B11.	(B)	databricks bundle destroy removes all resources defined in the bundle from the specified target workspace. Use with caution in production — it deletes jobs, pipelines, and other deployed resources.
B12.	(D)	DLT automatically manages checkpoints for streaming tables. When a pipeline restarts, it resumes from the last checkpoint — no reprocessing of already-processed files. This is one of the key advantages of DLT over raw Structured Streaming.
B13.	(C)	summary() returns all describe() statistics (count, mean, stddev, min, max) PLUS percentiles (25%, 50%, 75%). describe() does not compute percentiles because they require a full data scan.
B14.	(B)	Row filters are applied transparently at the query engine level. They cannot be seen or bypassed by regular users. The filter is pushed down into the scan automatically, and users see filtered results as if the filter does not exist.
B15.	(B)	Auto Loader checkpoints record the last committed offset. On restart with the same checkpointLocation, the stream resumes exactly where it left off — providing exactly-once semantics when writing to Delta Lake.

STRATEGY **PART 10 — Final Exam Strategy** Maximize your score on exam day

A. Time Budget by Domain

With 90 minutes and 45 questions, allocate your time based on domain weight and your confidence:

Domain	Questions	Time Budget	Focus If Weak
1 — Platform (6%)	~3	6 min	Delta time travel, VACUUM, compute types
2 — Ingestion (21%)	~9	18 min	COPY INTO idempotency, Auto Loader options, JDBC 4-option rule
3 — Transformation (22%)	~10	20 min	cast/null handling, joins, dedup, DLT expectations, ZORDER
4 — Lakeflow Jobs (16%)	~7	14 min	run_if values, task types (SQL=Warehouse), triggers
5 — CI/CD (10%)	~5	10 min	DABs mode:dev/prod, CLI commands, variable overrides
6 — Troubleshooting (10%)	~5	10 min	Spark UI skew, OOM types, small-file fix
7 — Governance (15%)	~6	12 min	3-privilege chain, managed vs external DROP, masking, RLS
Review buffer	—	10 min	Return to flagged questions

B. Top 20 Highest-Probability Exam Topics

Based on exam domain weights and exam guide emphasis, these topics have the highest probability of appearing:

Rank	Topic	Domain
1	COPY INTO idempotency and force=true	Domain 2
2	Auto Loader cloudFiles options and _rescued_data	Domain 2
3	Window ROW_NUMBER() for "keep latest" dedup	Domain 3
4	DLT expectations: expect vs expect_or_drop vs expect_or_fail	Domain 3
5	UC 3-privilege chain: USE CATALOG + USE SCHEMA + SELECT	Domain 7
6	Managed vs External table DROP behavior	Domain 7
7	run_if values: ALL_SUCCESS vs ALL_DONE vs NONE_FAILED	Domain 4
8	SQL task requires SQL Warehouse (not cluster)	Domain 4
9	Column masking: ALTER COLUMN SET MASK syntax	Domain 7
10	Row filter: SET ROW FILTER fn ON (col); BOOLEAN return	Domain 7
11	ZORDER vs Liquid Clustering differences	Domain 3
12	Left Semi vs Left Anti join distinctions	Domain 3
13	union() by position vs unionByName() by name	Domain 3
14	AQE is ON by default — auto-handles skew and coalescing	Domain 6
15	JDBC 4-option parallel read requirement	Domain 2
16	trigger(availableNow=True) replaces trigger(once=True)	Domain 2

17	mode:development auto-pauses schedules + prefixes names	Domain 5
18	databricks bundle validate vs deploy vs run	Domain 5
19	Data skew: one task >> others; AQE then salting	Domain 6
20	DENY overrides parent GRANT for specific principal	Domain 7

C. 12-Week Study Program Summary

Week	Phase	Topics Covered	Domain
1–2	Phase 1: Platform	Databricks architecture, Delta Lake ACID/time travel, Unity Catalog basics, compute types	D1
3	Phase 2: Ingestion	COPY INTO: syntax, idempotency, VALIDATE, format options	D2
4	Phase 2: Ingestion	Auto Loader: cloudFiles options, schema rescue/merge, checkpointing, triggers	D2
5	Phase 2: Ingestion	Lakeflow Connect (Standard/Managed), JDBC parallel, REST API, semi-structured data	D2
6	Phase 3: Transform	Bronze→Silver ETL: null handling, casting, column ops, explode, MERGE INTO	D3
7	Phase 3: Transform	All join types, broadcast, deduplication, aggregations, Window functions, union	D3
8	Phase 3: Transform	Gold layer objects, DLT expectations, Spark tuning, OPTIMIZE/ZORDER, Liquid Clustering	D3
9	Phase 4: Orchestration	Lakeflow Jobs: task types, DAG, run_if, retry, If/Else, For-Each, triggers	D4
10	Phase 4: CI/CD	Git Folders, DABs, databricks.yml, targets, mode dev/prod, CLI commands	D5
11a	Phase 5: Optimization	Spark UI, data skew, disk spill, OOM, Liquid Clustering, Predictive Optimization	D6
11b	Phase 5: Governance	Unity Catalog, GRANT/REVOKE/DENY, column masking, RLS, ABAC, lineage	D7
12	Final Prep	7-domain review, keyword map, 45-question mock exam, flash cards, exam strategy	All

■ **Note** You have completed a comprehensive 12-week study program covering all 7 exam domains with study guides, code examples, practice questions, and a full 45-question mock exam. You are ready.

D. Final Checklist Before the Exam

- ■ I can write COPY INTO with FORMAT_OPTIONS and COPY_OPTIONS from memory
- ■ I know cloudFiles.schemaLocation is required for Auto Loader inference
- ■ I understand trigger(availableNow=True) processes backlog then stops
- ■ I can write Window ROW_NUMBER() deduplication (keep latest per key)
- ■ I know all 7 join types and their how= values
- ■ I know DLT expect/expect_or_drop/expect_or_fail behaviors
- ■ I know the UC 3-privilege chain: USE CATALOG + USE SCHEMA + SELECT
- ■ I know managed DROP = data deleted; external DROP = data preserved
- ■ I can write GRANT, REVOKE, DENY statements
- ■ I know column mask (ALTER COLUMN SET MASK) and row filter (SET ROW FILTER) syntax
- ■ I know run_if ALL_DONE runs regardless of upstream outcome

- ■ I know SQL task requires SQL Warehouse — not a cluster
- ■ I know mode:development pauses schedules; mode:production needs run_as
- ■ I know bundle validate (syntax only), deploy (create resources), run (trigger job)
- ■ I can identify data skew in Spark UI and know AQE + salting fixes
- ■ I know Driver OOM = collect()/toPandas() on large DF → write() instead

GLOSSARY

Glossary

Key terms for the exam

Term	Definition
ACID	Atomicity, Consistency, Isolation, Durability — Delta Lake transaction guarantees
AQE	Adaptive Query Execution — runtime query optimization (auto-broadcast, coalesce, skew)
Auto Loader	cloudFiles streaming source for incremental file ingestion from cloud storage
ABAC	Attribute-Based Access Control — access decisions based on user + data attributes
Bronze/Silver/Gold	Medallion Architecture layers: raw / cleaned / business-aggregated data
Checkpoint	Auto Loader state store tracking which files are processed and stream offsets
CLUSTER BY	Liquid Clustering declaration at table creation — replaces ZORDER for new tables
COPY INTO	Idempotent SQL batch load command; tracks loaded files in Delta transaction log
DABs	Declarative Automation Bundles — define Databricks resources as YAML code
DLT	Delta Live Tables — declarative pipeline framework for streaming/batch ETL
External Location	UC object governing access to a cloud storage path via a storage credential
Job Cluster	Ephemeral cluster created for a job run and terminated after — cost-efficient
Lakehouse	Architecture combining data lake scale with data warehouse ACID/governance
Lakeflow Connect	Managed connector service for SaaS and database ingestion (Standard + Managed)
Liquid Clustering	Incremental data clustering via CLUSTER BY; replaces ZORDER for new tables
Materialized View	DLT-managed stored query result; auto-refreshed when upstream data changes
Medallion Architecture	Bronze → Silver → Gold data organization pattern
Metastore	Root UC object; one per cloud region; contains all catalogs
OPTIMIZE	Delta command to compact small files and optionally apply ZORDER clustering
Predictive Optimization	Auto-runs OPTIMIZE + VACUUM on UC managed tables based on usage patterns
Principal	UC identity: User, Service Principal, Group, or Account Group
run_if	Lakeflow Jobs task condition: ALL_SUCCESS/ALL_DONE/NONE_FAILED etc.
Shuffle	Spark data redistribution across executors — most expensive operation
Skew	Uneven data distribution across partitions; causes one task to take much longer
SQL Warehouse	Databricks compute for SQL queries and BI tools; required for SQL tasks in jobs
Time Travel	Delta feature to query historical table versions via VERSION/TIMESTAMP AS OF
Unity Catalog	Centralized governance layer: 3-level namespace, GRANT/REVOKE, lineage, audit
VACUUM	Delta command to permanently remove old data files beyond retention threshold
ZORDER BY	Delta clustering on filter columns for data skipping; full rewrite on each OPTIMIZE